



ระบบจัดการและวิเคราะห์การเงินส่วนบุคคลบนเว็บไซต์

โดย

นายชัชนันท์ หนูหริ่ง 660911074

นางสาวธัญญา พรตรี 660911078

เสนอ

ดร. ศักดิ์ระพี ไพศาลนันท์

รายงานนี้เป็นส่วนหนึ่งของรายวิชา 631498 เรื่องคัตเฉพาะทางวิศวกรรมไฟฟ้าสื่อสาร 2

ภาควิชาวิศวกรรมไฟฟ้า คณะวิศวกรรมศาสตร์และเทคโนโลยีอุตสาหกรรม

มหาวิทยาลัยศิลปากร ปีการศึกษา 2569

บทคัดย่อ

รายงานฉบับนี้นำเสนอผลการทดลองและการทดสอบระบบ MoneyMate ซึ่งเป็นเว็บแอปพลิเคชันสำหรับการจัดการรายรับ-รายจ่ายส่วนบุคคล พัฒนาในรูปแบบ Full-Stack Web Application โดย Frontend ใช้ HTML, CSS และ JavaScript และ Backend ใช้ Node.js และ Express.js การทดลองเน้นการออกแบบและเรียกใช้งาน REST API ของระบบเอง โดยใช้ Fetch API เป็นตัวกลางในการรับส่งข้อมูลระหว่างหน้าเว็บกับ Backend

การทดลองครอบคลุมการดึงข้อมูลด้วย GET การเพิ่มข้อมูลด้วย POST การแก้ไขข้อมูลด้วย PATCH การลบข้อมูลด้วย DELETE การกรองข้อมูลด้วย Query String รวมถึงการทดสอบกรณีข้อมูลไม่ครบและกรณีไม่พบ Transaction ระบบใช้ JSON เป็นรูปแบบข้อมูล และจัดเก็บ Transaction หลักไว้ที่ server/transactions.json ตามรายละเอียดใน README.md

จากผลการทดลองที่ระบุใน README.md ระบบตอบสนองตาม HTTP Status Code ที่ออกแบบไว้ ได้แก่ 200 OK สำหรับ GET/PATCH, 201 Created สำหรับ POST, 204 No Content สำหรับ DELETE, 400 Bad Request สำหรับข้อมูลไม่ถูกต้องหรือไม่ครบ และ 404 Not Found สำหรับรายการหรือ Endpoint ที่ไม่พบ นอกจากนี้ Frontend สามารถนำข้อมูลที่ได้จาก API ไปแสดงผลและอัปเดตข้อมูลบนหน้าเว็บโดยไม่ต้องโหลดหน้าใหม่

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญ

MoneyMate เป็นเว็บแอปพลิเคชันสำหรับการจัดการรายรับ-รายจ่ายส่วนบุคคล มีเป้าหมายเพื่อช่วยให้ผู้ใช้สามารถเพิ่ม แก้ไข ลบ ค้นหา และกรองรายการธุรกรรม รวมทั้งดูภาพรวมทางการเงินผ่าน Dashboard และ Charts ได้จากระบบเดียว

โครงการพัฒนาในรูปแบบ Full-Stack Web Application โดย Frontend ติดต่อกับ Backend ผ่าน REST API ของระบบเอง ข้อมูล Transaction ถูกจัดการผ่าน Endpoint ของ /api/transactions และจัดเก็บข้อมูลหลักใน server/transactions.json

การทดลองในรายงานฉบับนี้จึงเน้นการตรวจสอบการทำงานของ REST API และการเชื่อมต่อกับหน้าเว็บตามที่อธิบายไว้ใน README.md โดยไม่ขยายขอบเขตไปยังฟังก์ชันหรือเทคโนโลยีที่ไม่ได้ระบุไว้ในเอกสารโครงการ

1.2 วัตถุประสงค์ของการทดลอง

- ทดสอบการดึงข้อมูล Transaction ด้วย GET
- ทดสอบการเพิ่ม Transaction ด้วย POST
- ทดสอบการแก้ไข Transaction ด้วย PATCH
- ทดสอบการลบ Transaction ด้วย DELETE
- ทดสอบ Query String สำหรับกรองข้อมูล
- ทดสอบ Validation และ Error Handling
- ตรวจสอบการเชื่อมต่อระหว่าง Frontend กับ Backend ผ่าน Fetch API

1.3 ขอบเขตของการทดลอง

หัวข้อ	ขอบเขต
API	GET, POST, PATCH, DELETE
Query	type, category, month, year
Validation	ข้อมูลไม่ครบ/ไม่ถูกต้อง
Error	400 Bad Request และ 404 Not Found
Storage	server/transactions.json
Frontend	การเรียก API ด้วย Fetch API
เครื่องมือทดสอบ	Thunder Client ตาม README.md

1.4 ประโยชน์ที่คาดว่าจะได้รับ

- เข้าใจการออกแบบ REST API ของระบบ
- เข้าใจการรับส่งข้อมูลระหว่าง Frontend และ Backend
- ตรวจสอบพฤติกรรมของ HTTP Methods และ Status Codes
- สามารถนำแนวทางไปใช้กับการพัฒนาเว็บแอปพลิเคชันอื่นได้

บทที่ 2

ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง

2.1 REST API

REST API เป็นรูปแบบการสื่อสารระหว่าง Client และ Server โดย MoneyMate กำหนด Resource หลักเป็น transactions และใช้ HTTP Method เพื่อกำหนดการทำงาน เช่น GET สำหรับอ่านข้อมูล POST สำหรับเพิ่มข้อมูล PATCH สำหรับแก้ไขข้อมูล และ DELETE สำหรับลบข้อมูล

2.2 Node.js และ Express.js

Node.js และ Express.js ถูกใช้เป็นส่วน Backend ของระบบ โดย Express ทำหน้าที่รับ Request จาก Frontend และจัดการ Endpoint ของ REST API ก่อนส่ง Response กลับไปยัง Client

2.3 Fetch API

Fetch API ถูกใช้ใน JavaScript เพื่อเรียก REST API ของระบบ โดย Frontend สามารถเรียก GET เพื่อดึงข้อมูล และใช้ POST, PATCH หรือ DELETE เพื่อเปลี่ยนแปลงข้อมูลผ่าน Request ที่ส่งไปยัง Backend

2.4 JSON

JSON ใช้เป็นรูปแบบข้อมูลของ Transaction และเป็นรูปแบบข้อมูลที่ Backend รับและส่งระหว่าง Frontend กับ API โดยตัวอย่าง Transaction มีข้อมูล id, title, amount, category, type และ date

2.5 LocalStorage

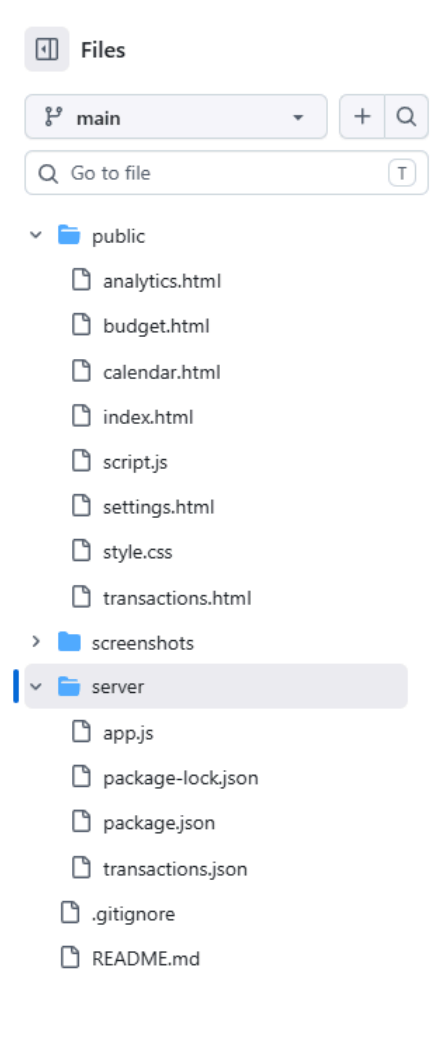
README.md ระบุว่า LocalStorage ใช้สำหรับข้อมูลหรือการตั้งค่าฝั่ง Client เช่น Theme, Budget & Goals, Animation Preference, Notification Preference และ Demo Data ขณะที่ Transaction หลักใช้ข้อมูลจาก Backend เมื่อ Server สามารถเชื่อมต่อได้

2.6 เทคโนโลยีที่ใช้ในโครงการ

ส่วน	เทคโนโลยี
Frontend	HTML5, CSS3, JavaScript, Fetch API, Chart.js, Phosphor Icons, LocalStorage
Backend	Node.js, Express.js, REST API, JSON, Node.js File System (fs)
Development	Visual Studio Code, npm, Git, GitHub, Thunder Client, Browser Developer Tools

2.7 โครงสร้างโครงการ

```
Mini-project-MoneyMate/  
├── public/  
│   ├── index.html  
│   ├── transactions.html  
│   ├── analytics.html  
│   ├── budget.html  
│   ├── calendar.html  
│   ├── settings.html  
│   ├── style.css  
│   └── script.js  
├── server/  
│   ├── app.js  
│   ├── package.json  
│   ├── package-lock.json  
│   └── transactions.json  
└── screenshots/
```



บทที่ 3

การออกแบบระบบและ REST API

3.1 ภาพรวมการทำงาน

User
↓
Frontend (HTML / CSS / JavaScript)
↓ Fetch API
Express Server
↓
REST API /api/transactions
↓
server/transactions.json

3.2 Resource และ Base URL

Resource หลักของระบบคือ transactions และ Base URL ของ REST API คือ
<http://localhost:3000/api>

3.3 โครงสร้างข้อมูล Transaction

Field	ตัวอย่าง	หน้าที่
id	1	รหัสรายการ
title	Coffee	ชื่อรายการ
amount	80	จำนวนเงิน
category	Food	หมวดหมู่
type	expense	ประเภท income หรือ expense
date	2026-09-24	วันที่

3.4 รายการ Endpoint

Method	Endpoint	หน้าที่	Status
GET	/api/transactions	ดึงข้อมูลทั้งหมด	200 OK
GET	/api/transactions/:id	ดึงข้อมูลตาม ID	200 / 404
GET	/api/transactions? type=expense	กรองตาม Type	200 OK
GET	/api/transactions? category=Food	กรองตาม Category	200 OK
GET	/api/transactions? month=9	กรองตาม Month	200 OK
GET	/api/transactions? year=2026	กรองตาม Year	200 OK
POST	/api/transactions	เพิ่มรายการ	201 Created
PATCH	/api/transactions/:id	แก้ไขรายการ	200 OK
DELETE	/api/transactions/:id	ลบรายการ	204 No Content

3.5 Validation

Backend ตรวจสอบข้อมูลก่อนเพิ่มหรือแก้ไข Transaction โดยข้อมูลหลัก ได้แก่ title, amount, category, type และ date เงื่อนไขที่ระบุใน README.md คือ title ต้องไม่ว่าง amount ต้องเป็นตัวเลขมากกว่า 0 category ต้องเป็นหมวดหมู่ที่รองรับ type ต้องเป็น income หรือ expense และ date ต้องอยู่ในรูปแบบ YYYY-MM-DD

3.6 Error Handling

กรณี	Status	ผลลัพธ์
ข้อมูลไม่ครบ/ไม่ถูกต้อง	400 Bad Request	ส่งข้อความแจ้งข้อผิดพลาด
ไม่พบ Transaction	404 Not Found	Transaction not found
GET/PATCH สำเร็จ	200 OK	ส่งข้อมูลกลับ
POST สำเร็จ	201 Created	สร้างรายการใหม่
DELETE สำเร็จ	204 No Content	ไม่มี Response Body

บทที่ 4

ผลการทดลองและการทดสอบระบบ

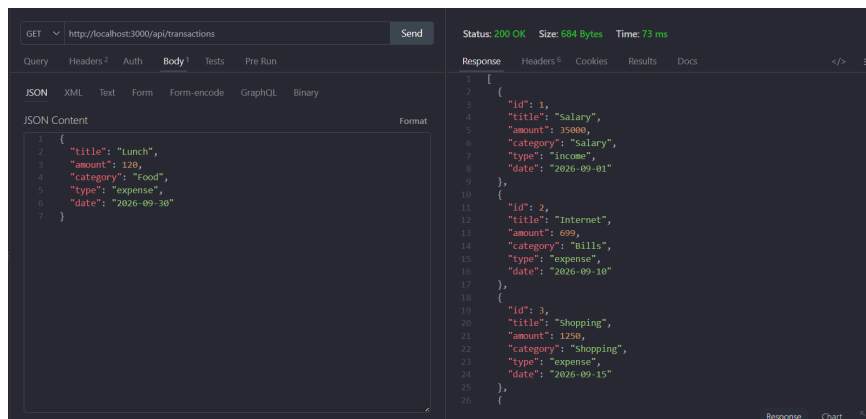
4.1 สภาพแวดล้อมและขั้นตอนการรัน

ตาม README.md สามารถติดตั้งและรันระบบได้โดย Clone Repository จาก GitHub จากนั้นเข้าโฟลเดอร์ server ติดตั้ง Dependencies ด้วย npm install และรันด้วย npm run dev หรือ npm start เมื่อ Server ทำงานสำเร็จจะให้บริการที่ http://localhost:3000

```
git clone https://github.com/Yin-Yew/Mini-project-MoneyMate.git
cd Mini-project-MoneyMate
cd server
npm install
npm run dev
# หรือ
npm start
```

4.2 การทดสอบการดึงข้อมูล (GET)

การทดสอบ GET ใช้สำหรับดึงรายการ Transaction ทั้งหมดจาก /api/transactions โดย README.md ระบุผลการทดสอบเป็น 200 OK



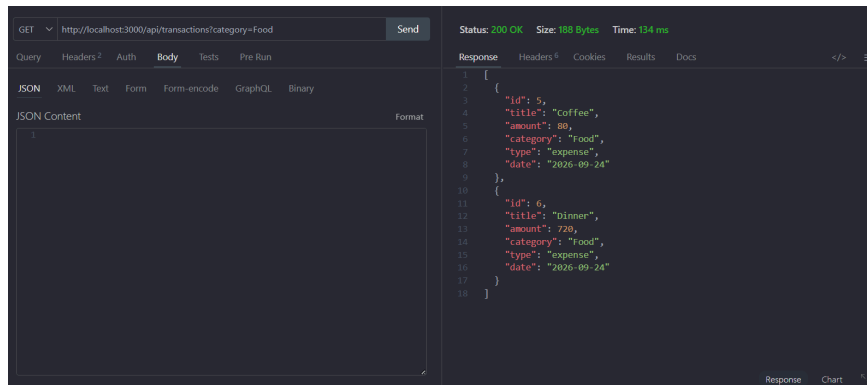
GET http://localhost:3000/api/transactions

ผลการทดลอง: ระบบสามารถส่งรายการ Transaction กลับมาในรูปแบบ JSON และตอบสถานะ 200 OK ตามข้อมูลใน README.md

4.3 การทดสอบ Query String

ระบบรองรับการกรอกข้อมูลด้วย Query String เช่น type, category, month และ year ตัวอย่างการกรอกตาม Category คือ GET /api/transactions?category=Food

GET <http://localhost:3000/api/transactions?category=Food>



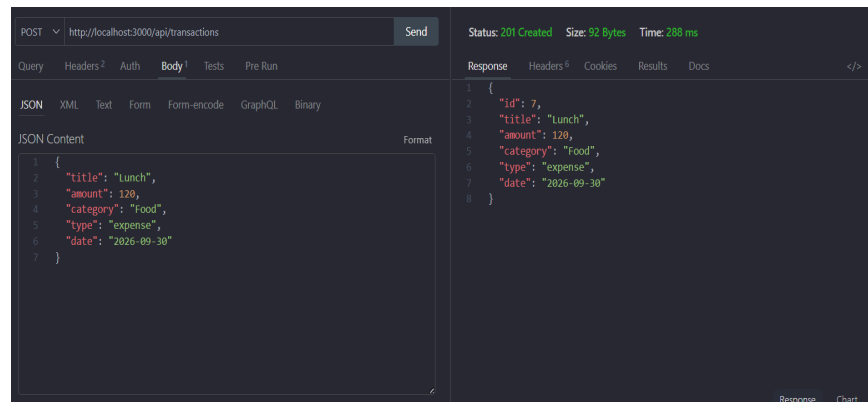
ผลการทดลอง: ระบบส่งกลับเฉพาะ Transaction ที่ตรงกับเงื่อนไข Category ที่ระบุ และตอบสถานะ 200 OK ตาม README.md

4.4 การทดสอบการเพิ่มข้อมูล (POST)

การทดสอบ POST ใช้สำหรับเพิ่ม Transaction ใหม่ โดยส่งข้อมูล title, amount, category, type และ date ใน Request Body

POST /api/transactions

```
{
  "title": "Lunch",
  "amount": 120,
  "category": "Food",
  "type": "expense",
  "date": "2026-09-30"
}
```



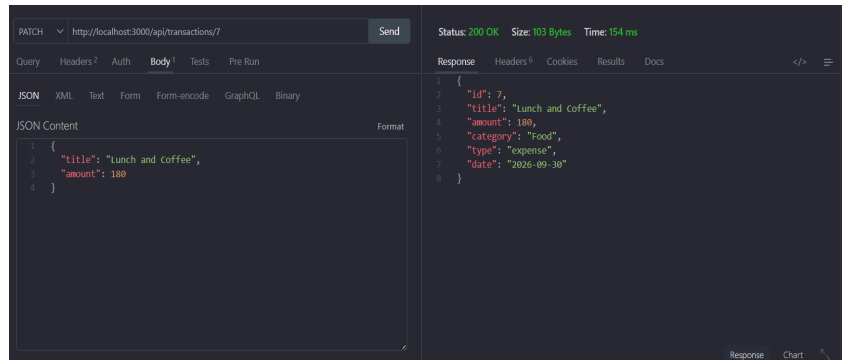
ผลการทดลอง: ระบบสร้าง Transaction ใหม่และตอบสถานะ 201 Created ตาม README.md

4.5 การทดสอบการแก้ไขข้อมูล (PATCH)

การทดสอบ PATCH ใช้สำหรับแก้ไขข้อมูล Transaction ตาม ID โดยสามารถส่งเฉพาะข้อมูลที่ต้องการแก้ไขได้

PATCH /api/transactions/7

```
{
  "title": "Lunch and Coffee",
  "amount": 180
}
```

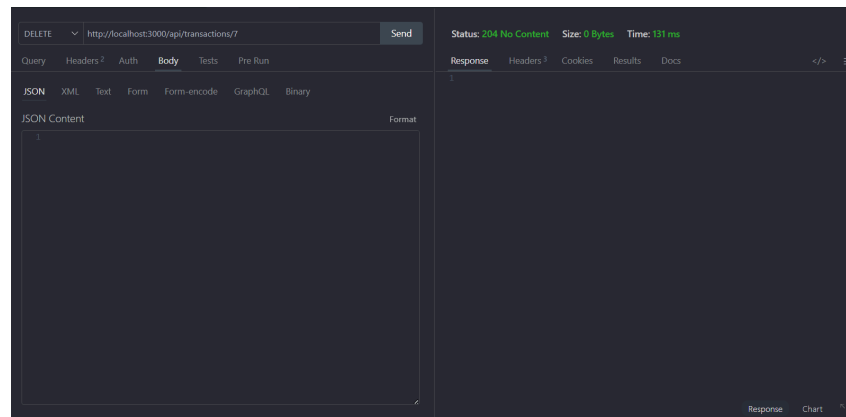


ผลการทดลอง: ระบบแก้ไขข้อมูล Transaction ตาม ID และตอบสถานะ 200 OK ตาม README.md

4.6 การทดสอบการลบข้อมูล (DELETE)

การทดสอบ DELETE ใช้สำหรับลบ Transaction ตาม ID

DELETE /api/transactions/7

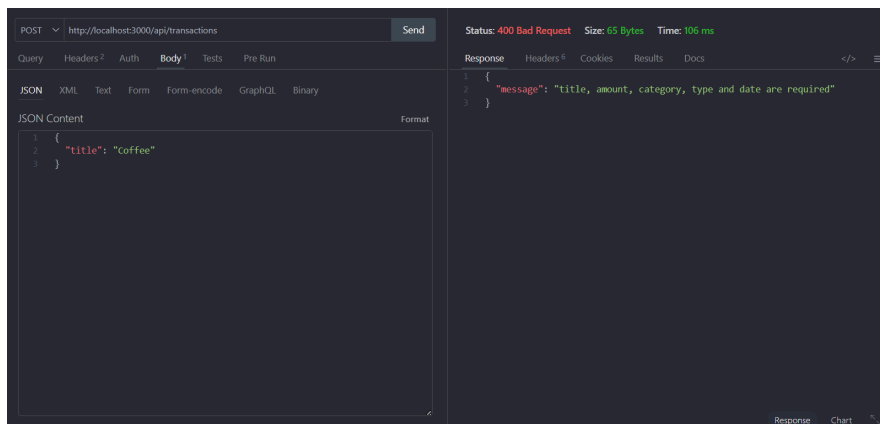


ผลการทดลอง: ระบบลบ Transaction สำเร็จและตอบสถานะ 204 No Content โดยไม่มี Response Body ตาม [README.md](#)

4.7 การทดสอบ Validation: 400 Bad Request

ทดสอบโดยส่งข้อมูลไม่ครบ เช่นส่งเฉพาะ title

```
{  
  "title": "Coffee"  
}
```

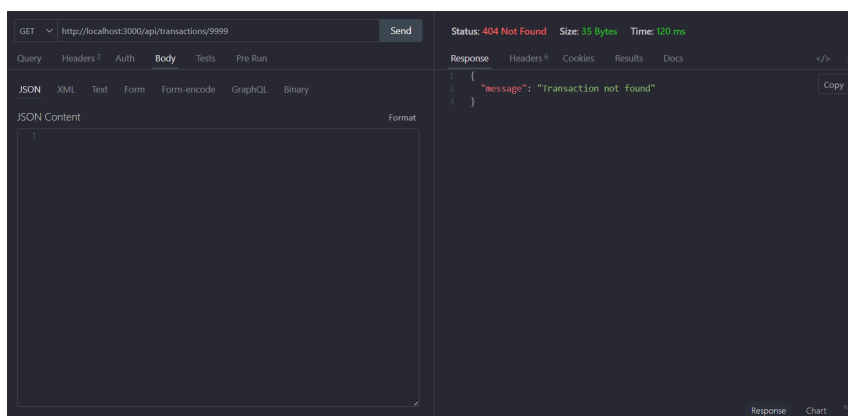


ผลการทดลอง: ระบบปฏิเสธ Request เนื่องจากข้อมูลที่ไม่จำเป็นไม่ครบ และตอบสถานะ 400 Bad Request

4.8 การทดสอบกรณีไม่พบข้อมูล: 404 Not Found

ทดสอบโดยเรียก Transaction ID ที่ไม่มีอยู่ เช่น 9999

GET `http://localhost:3000/api/transactions/9999`



ผลการทดลอง: ระบบแจ้งว่าไม่พบ Transaction และตอบสถานะ 404 Not Found

4.9 สรุปผลการทดสอบ REST API

รายการทดสอบ	ผลที่ระบุใน README.md
GET	200 OK
GET + Query String	200 OK
POST	201 Created
PATCH	200 OK
DELETE	204 No Content
Invalid Request	400 Bad Request
Unknown ID	404 Not Found

บทที่ 5

โครงสร้าง HTML

1. ภาพรวมไฟล์ HTML

จากโครงสร้างโครงการใน README.md ส่วน Frontend อยู่ในโฟลเดอร์ public/ และมีไฟล์ HTML หลัก 6 หน้า ได้แก่ index.html, transactions.html, analytics.html, budget.html, calendar.html และ settings.html แต่ละไฟล์ทำหน้าที่เป็นหน้าเว็บคนละส่วนของ MoneyMate และใช้ style.css กับ script.js ร่วมกัน

ไฟล์	หน้าที่
index.html	Dashboard — ภาพรวมทางการเงิน
transactions.html	จัดการรายการรายรับและรายจ่าย
analytics.html	วิเคราะห์ข้อมูลทางการเงิน
budget.html	สร้างและติดตามเป้าหมายการออม
calendar.html	ดู Transaction ตามวันที่
settings.html	ตั้งค่า Theme และ Preferences

2. HTML Tag ที่ใช้ร่วมกัน

องค์ประกอบ	การทำงานในโครงการ
<html>	โครงสร้างหลักของเอกสาร HTML และกำหนดภาษาเป็น th
<head>	เก็บ charset, viewport, title และไฟล์ CSS/Library
<body>	เก็บเนื้อหาที่ผู้ใช้เห็น
<aside>	พื้นที่ Sidebar และเมนู
<nav>	ลิงก์เปลี่ยนหน้าในระบบ
<main>	พื้นที่เนื้อหาหลักของแต่ละหน้า
<header>	ชื่อหน้าและคำอธิบาย
<section>	แบ่งเนื้อหาเป็นส่วน ๆ
<div>	จัดกลุ่มองค์ประกอบ
<h1>/<h2>/<h3>	หัวข้อระดับต่าง ๆ
<p>	ข้อความ/คำอธิบาย
<button>	ปุ่มที่สามารถเรียก JavaScript
<input>	รับข้อมูลจากผู้ใช้
<select>/<option>	ช่องเลือกข้อมูล
<canvas>	พื้นที่ให้ Chart.js สร้างกราฟ
<a>	ลิงก์ไปยังหน้าอื่น
<script>	โหลด JavaScript และ Library

3. index.html — Dashboard

index.html เป็นหน้า Dashboard ใช้สรุปภาพรวมทางการเงินของผู้ใช้

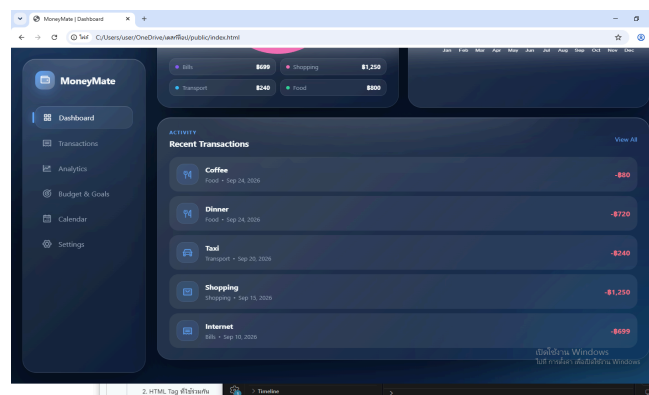
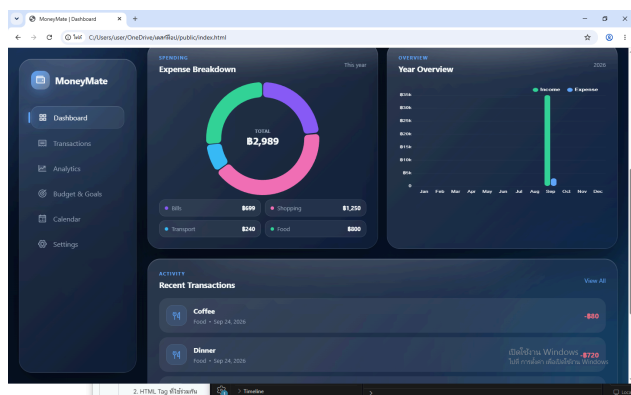
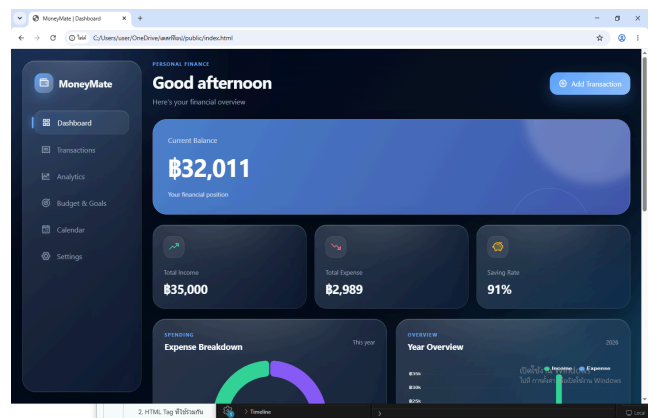
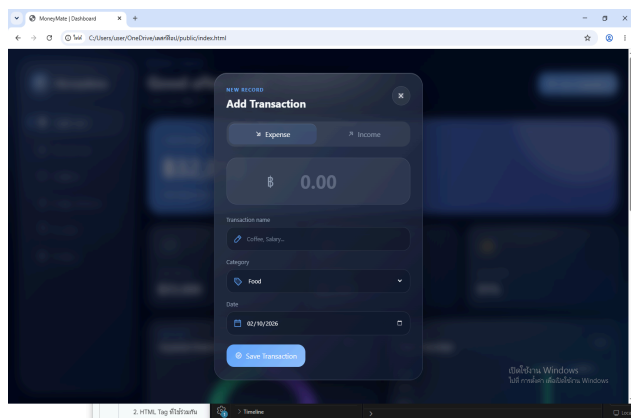
- Sidebar ใช้นำทางไปยังทั้ง 6 หน้า
- header แสดงชื่อ Dashboard และปุ่ม Add Transaction
- balance แสดง Current Balance
- cards แสดง Total Income, Total Expense และ Saving Rate
- expenseChart เป็นพื้นที่กราฟ Expense Breakdown
- yearChart เป็นพื้นที่กราฟ Year Overview
- dashboardRecentTransactions เป็นตำแหน่งแสดงรายการล่าสุด
- addModal เป็นหน้าต่างสำหรับเพิ่ม Transaction

```
<canvas id="expenseChart"></canvas>
```

```
<canvas id="yearChart"></canvas>
```

```
<div id="dashboardRecentTransactions"></div>
```

HTML สร้างโครงสร้างและพื้นที่แสดงผล ส่วน JavaScript จะนำข้อมูลมาใส่ในองค์ประกอบเหล่านี้ และเรียก Chart.js เพื่อสร้างกราฟ



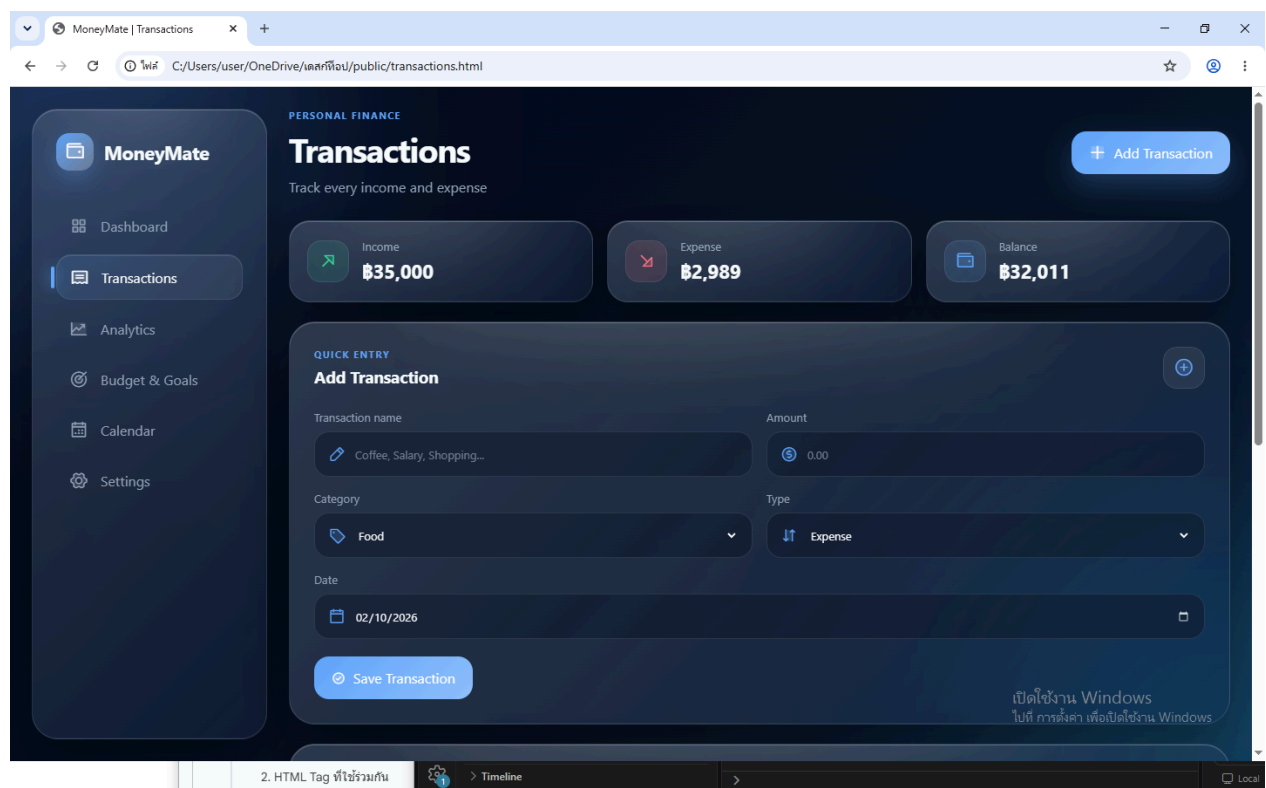
4. transactions.html — Transactions

transactions.html เป็นหน้าหลักสำหรับจัดการ Transaction มีทั้งการเพิ่ม ค้นหา กรอง และแก้ไขข้อมูล

- transaction-summary แสดง Income, Expense และ Balance
- addTransactionSection เป็นแบบฟอร์มเพิ่มรายการ
- title รับชื่อรายการ
- amount รับจำนวนเงิน
- category เลือกหมวดหมู่
- type เลือก Income/Expense
- date เลือกวันที่
- transaction-list แสดงรายการทั้งหมด
- searchInput ใช้ค้นหา
- filterType ใช้กรอง Income/Expense
- editModal ใช้แก้ไข Transaction

```
<input id="title" type="text">
<input id="amount" type="number">
<select id="category">...</select>
<select id="type">...</select>
<input id="date" type="date">
```

เมื่อกดปุ่มบันทึก JavaScript จะรับค่าจาก Input และเรียก API เพื่อเพิ่มหรือแก้ไขข้อมูล จากนั้นนำผลลัพธ์กลับมาแสดงบนหน้าเว็บ



5. analytics.html — Analytics

analytics.html ใช้แสดงผลการวิเคราะห์ข้อมูลทางการเงิน

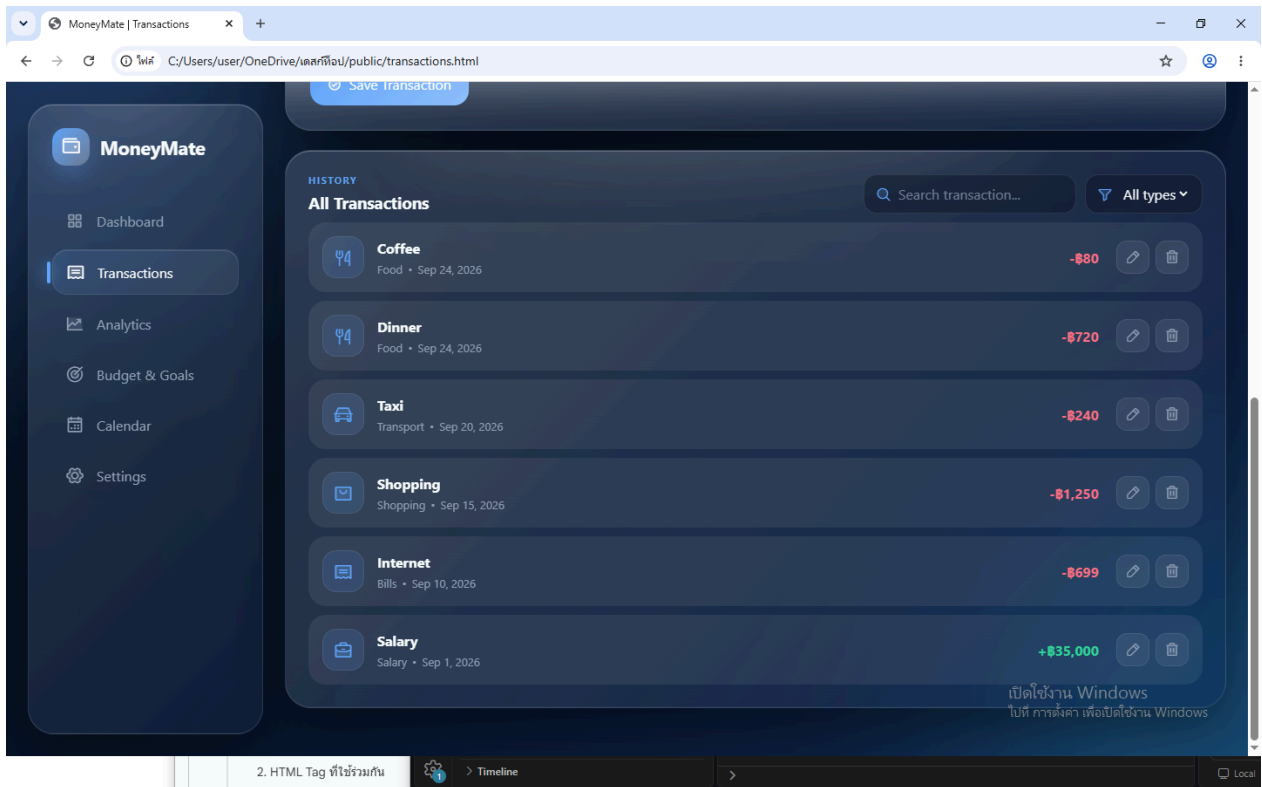
- cards แสดง Total Expense, Saving Rate และ Average Expense
- categoryChart ใช้แสดง Category Breakdown
- trendChart ใช้แสดง Monthly Expense
- analyticsInsight ใช้แสดง Financial Insight

```
<canvas id="categoryChart"></canvas>
```

```
<canvas id="trendChart"></canvas>
```

```
<p id="analyticsInsight">Your spending insight will appear here.</p>
```

HTML ทำหน้าที่เตรียมพื้นที่แสดงผล ส่วน JavaScript และ Chart.js ทำหน้าที่นำข้อมูล Transaction มาคำนวณและสร้างกราฟ



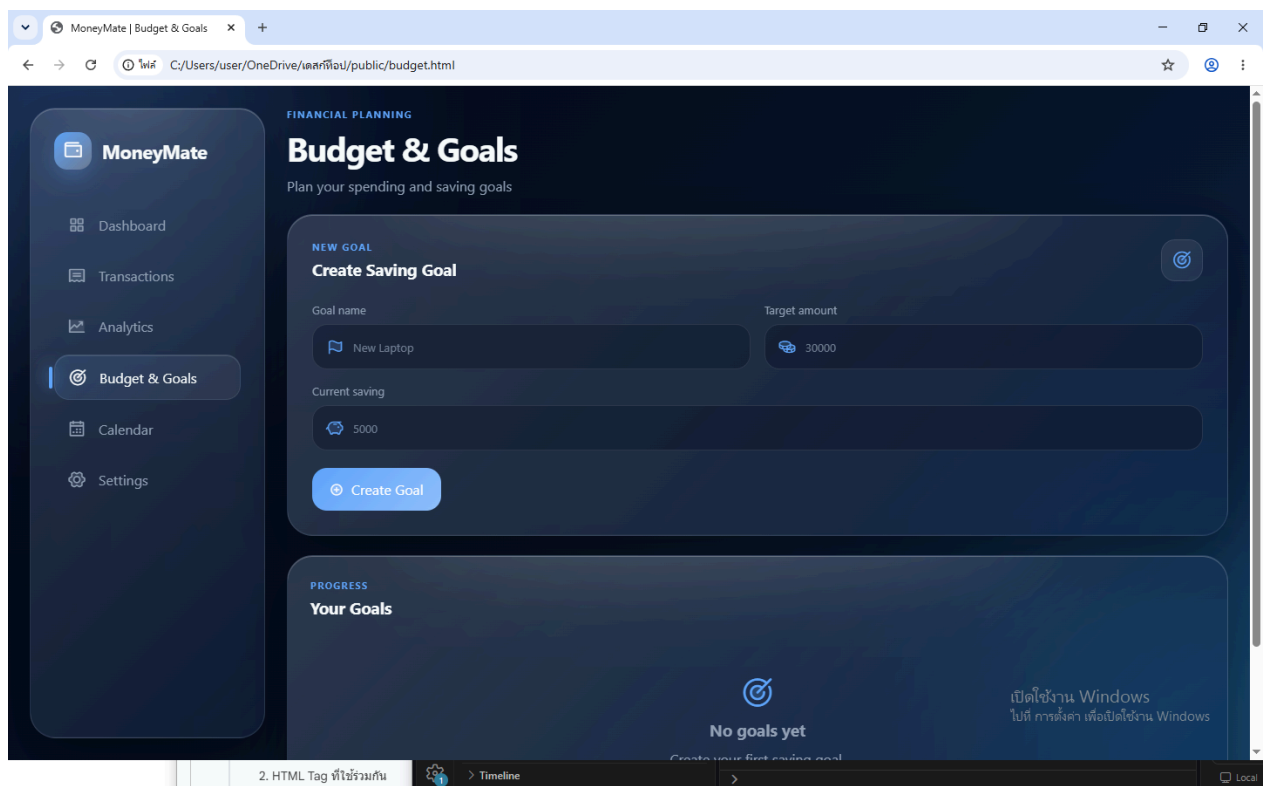
6. budget.html — Budget & Goals

budget.html ใช้สร้างและติดตาม Saving Goal โดย README ระบุว่าเวอร์ชันนี้เน้น Saving Goals และยังไม่มี Monthly Budget แยกตาม Category

- goalName รับชื่อเป้าหมาย
- goalAmount รับ Target Amount
- goalCurrent รับ Current Saving
- ปุ่ม Create Goal เรียก addGoal()
- goalList แสดงรายการเป้าหมาย
- savingModal ใช้เพิ่มเงินออม
- savingAmount รับจำนวนเงินที่จะเพิ่ม
- ปุ่ม Add Saving เรียก confirmAddSaving()

ข้อมูล Goal ใช้ LocalStorage ตาม README.md ส่วน JavaScript คำนวณ Progress และนำข้อมูลมาแสดงใน goalList

```
<input id="goalName" type="text">
<input id="goalAmount" type="number">
<input id="goalCurrent" type="number">
<div id="goalList"></div>
```



7. calendar.html — Calendar

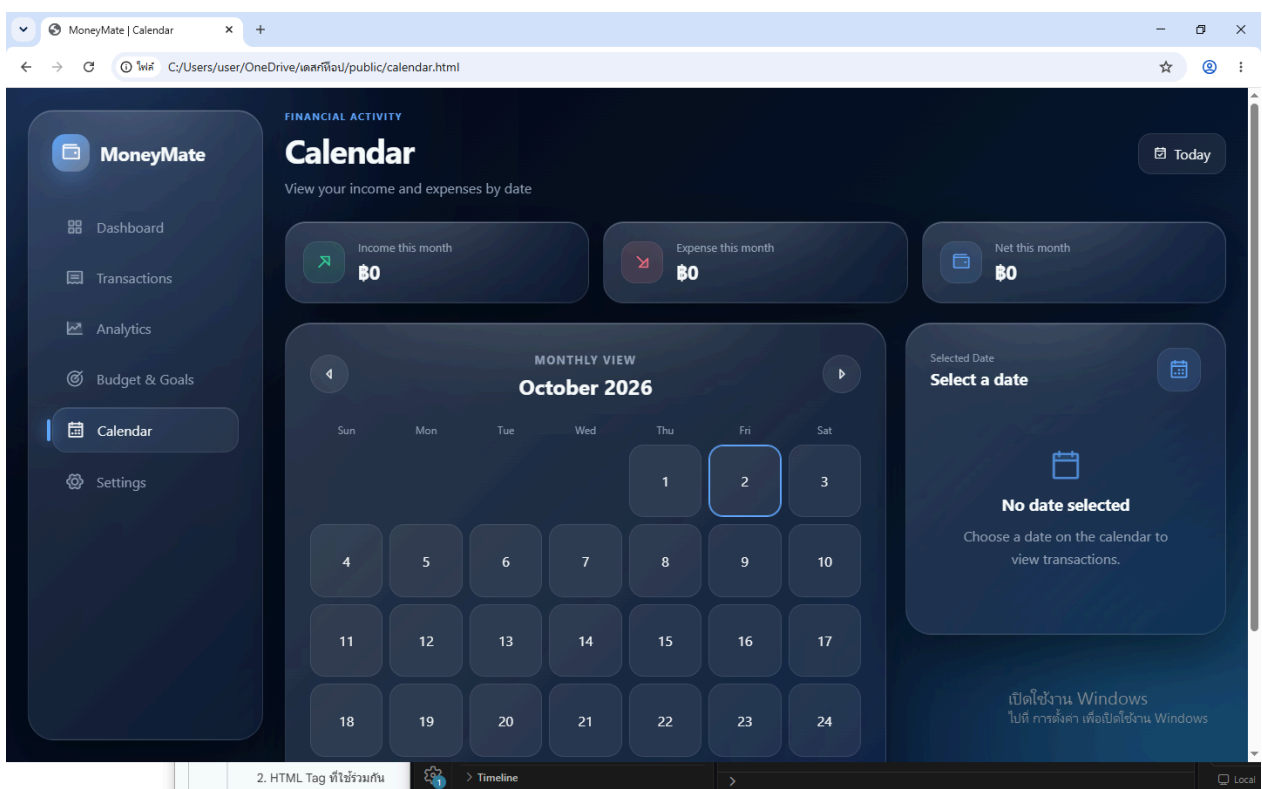
calendar.html ใช้ดูรายรับและรายจ่ายตามวันที่ สามารถเปลี่ยนเดือน กลับไปวันนี้ และเลือกวันที่เพื่อดู Transaction

- calendar-stats แสดง Income this month, Expense this month และ Net this month
- calendarMonth แสดงเดือนปัจจุบัน
- ปุ่ม Previous/Next เรียก changeMonth(-1) และ changeMonth(1)
- calendarGrid เป็นพื้นที่สร้างปฏิทิน
- selectedDateTitle แสดงวันที่เลือก
- calendarTransactionList แสดง Transaction ของวันที่เลือก
- ปุ่ม Today เรียก goToToday()

```
<div id="calendarGrid" class="calendar-grid"></div>
```

```
<h3 id="selectedDateTitle">Select a date</h3>
```

```
<div id="calendarTransactionList"></div>
```



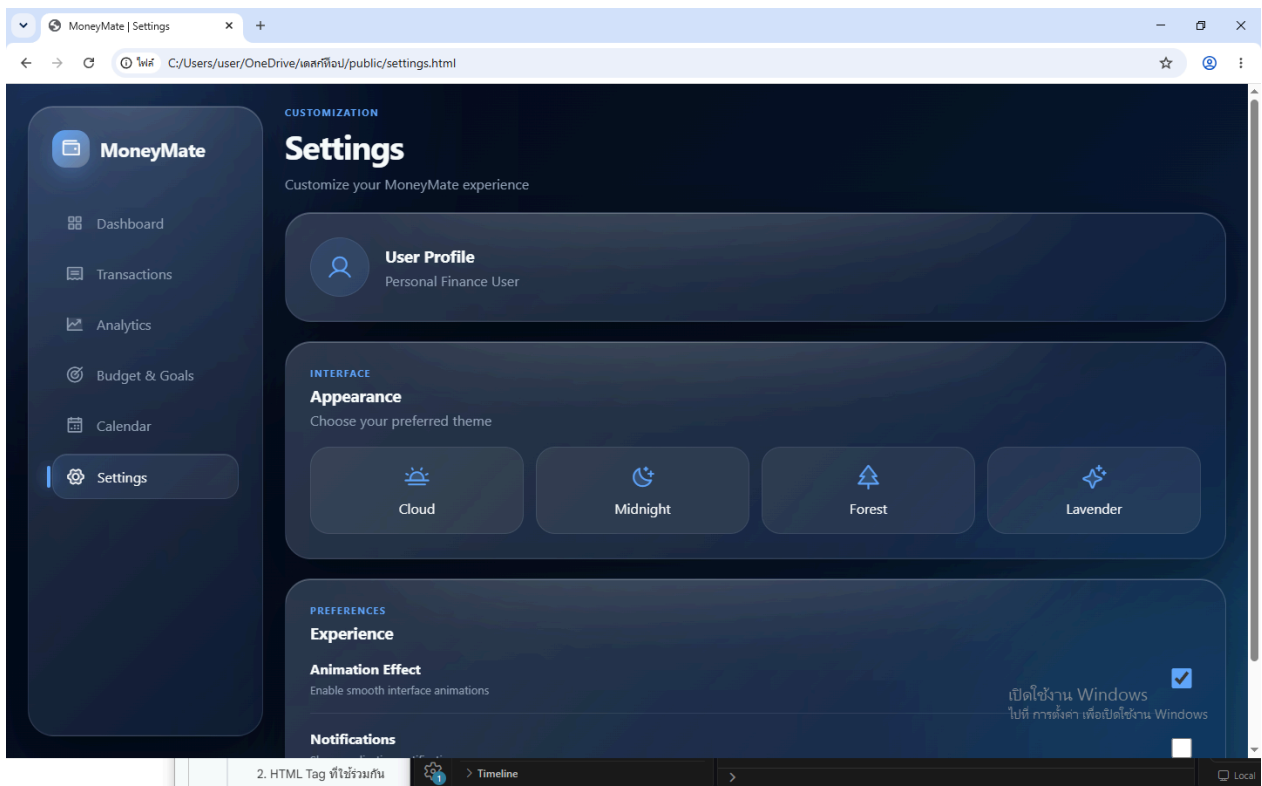
8. settings.html — Settings

settings.html ใช้ตั้งค่าหน้าตาและ Preferences ของระบบ

- User Profile แสดงข้อมูลโปรไฟล์พื้นฐาน
- Appearance มี Theme 4 แบบ: Cloud, Midnight, Forest และ Lavender
- ปุ่ม Theme เรียก changeTheme()
- Animation Effect เป็น Checkbox สำหรับ Animation
- Notifications เป็น Checkbox สำหรับ Notifications
- savePreferences() ทำงานเมื่อมีการเปลี่ยน Preference

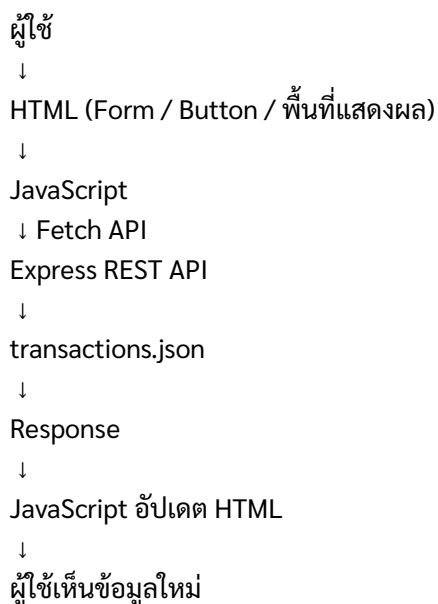
```
<button onclick="changeTheme('cloud')">Cloud</button>
<button onclick="changeTheme('midnight')">Midnight</button>
<button onclick="changeTheme('forest')">Forest</button>
<button onclick="changeTheme('lavender')">Lavender</button>
```

README.md ระบุว่า Theme และ Preferences บางส่วนถูกจัดเก็บด้วย LocalStorage



9. การทำงานร่วมกันของ HTML, CSS และ JavaScript

HTML ทำหน้าที่สร้างโครงสร้างและตำแหน่งขององค์ประกอบ CSS ทำหน้าที่ตกแต่งและจัด Layout ส่วน JavaScript ทำหน้าที่ควบคุมการทำงาน รับค่าจากผู้ใช้ เรียก REST API และอัปเดตข้อมูลบนหน้าเว็บ



ตัวอย่างเช่นใน transactions.html ผู้ใช้กรอกข้อมูลลง Form จากนั้นกด Save Transaction JavaScript จะรับค่าและส่ง POST ไปยัง /api/transactions เมื่อ Backend ตอบกลับ JavaScript จะนำข้อมูลมาแสดงในรายการโดยไม่จำเป็นต้องโหลดหน้าใหม่

10. สรุป

ไฟล์	หน้าที่หลัก	องค์ประกอบเด่น
index.html	Dashboard	Balance, Cards, Charts, Recent Transactions, Add Modal
transactions.html	จัดการ Transaction	Form, Search, Filter, List, Edit Modal
analytics.html	วิเคราะห์ข้อมูล	Summary Cards, Category Chart, Trend Chart, Insight
budget.html	Saving Goals	Goal Form, Goal List, Saving Modal
calendar.html	Transaction ตามวันที่	Calendar Grid, Month Navigation, Date Detail
settings.html	ตั้งค่าระบบ	Theme Buttons, Profile, Preferences